

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Основы программной инженерии

Лабораторная работа №4

Вариант 408602

Выполнила:

Павличенко Софья Алексеевна, Р3215

Проверила:

Харитоновна Анастасия Евгеньевна

Санкт-Петербург 2025г.

Оглавление

Задание	3
Решение.....	4
Задание 1	4
Задание 2	6
Задание 3	9
Задание 4	11
Заключение	18

Задание

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить наименование и версию JVM, поставщика виртуальной машины Java и номер её сборки.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Решение

Задание 1

PointStats.kt

```
package mbeans

import entity.Point
import javax.management.Notification
import javax.management.NotificationBroadcasterSupport

interface PointStatsMBean {
    fun addPoint(point: Point)
    fun getTotalPoints(): Int
    fun getOutsidePoints(): Int
}

class PointStats: NotificationBroadcasterSupport(), PointStatsMBean {

    private var totalPoints = 0
    private var outsidePoints = 0
    var sequence = 1L

    override fun addPoint(point: Point) {
        totalPoints++
        if (!point.inArea) {
            outsidePoints++
            val notification: Notification = Notification(
                "point.outside", this, sequence++,
                System.currentTimeMillis(),
                "Point is outside: (${point.x}, ${point.y})"
            )
            sendNotification(notification)
        }
    }

    override fun getTotalPoints(): Int = totalPoints

    override fun getOutsidePoints(): Int = outsidePoints
}
```

ClickStats.kt

```
package mbeans

interface ClickStatsMBean {
    fun registerClick()
    fun getAverageClickInterval(): Double
}

class ClickStats: ClickStatsMBean {
    var lastClickTime = -1L
    var totalInterval = 0.0
    var clickCount = 0

    override fun registerClick() {
```

```

        val currentTime = System.currentTimeMillis()
        if (lastClickTime != -1L) {
            totalInterval += currentTime - lastClickTime
            clickCount++
        }
        lastClickTime = currentTime
    }

    override fun getAverageClickInterval(): Double {
        return if (clickCount == 0) 0.0 else totalInterval / clickCount
    }
}

```

MBeanRegistrar.kt

```

package mbeans

import jakarta.servlet.ServletContextEvent
import jakarta.servlet.ServletContextListener
import java.lang.management.ManagementFactory
import javax.management.MBeanServer
import javax.management.ObjectName

class MBeanRegistrar : ServletContextListener {

    override fun contextInitialized(sce: ServletContextEvent?) {
        val mbs: MBeanServer = ManagementFactory.getPlatformMBeanServer()
        val pointStats = PointStats()
        val clickStats = ClickStats()

        val pointStatsName = ObjectName("app.monitoring:type=PointStats")
        val clickStatsName = ObjectName("app.monitoring:type=ClickStats")

        if (!mbs.isRegistered(pointStatsName)) {
            mbs.registerMBean(pointStats, pointStatsName)
        }

        if (!mbs.isRegistered(clickStatsName)) {
            mbs.registerMBean(clickStats, clickStatsName)
        }
    }
}

```

Задание 2

Метаданные MBean'ов:

The screenshot shows the JBoss JMX console with the 'MBeans' tab selected. The left sidebar displays a tree of MBeans, with 'ClickStats' under 'app.monitoring' highlighted. The main panel shows the metadata for 'ClickStats'.

Name	Value
Info:	
ObjectName	app.monitoring:type=ClickStats
ClassName	mbeans.ClickStats
Description	Information on the management interface of the MBean
Constructor-0:	
Name	mbeans.ClickStats
Description	Public constructor of the MBean

Name	Value
Descriptor	
Info:	
immutableInfo	true
interfaceClassName	mbeans.ClickStatsMBean
mxbean	false

The screenshot shows the JBoss JMX console with the 'MBeans' tab selected. The left sidebar displays a tree of MBeans, with 'PointStats' under 'app.monitoring' highlighted. The main panel shows the metadata for 'PointStats'.

Name	Value
Info:	
ObjectName	app.monitoring:type=PointStats
ClassName	mbeans.PointStats
Description	Information on the management interface of the MBean
Constructor-0:	
Name	mbeans.PointStats
Description	Public constructor of the MBean

Name	Value
Descriptor	
Info:	
immutableInfo	true
interfaceClassName	mbeans.PointStatsMBean
mxbean	false

Атрибуты MBean'ов:

pid: 13570 /Users/sonikpav/ITMO/fundamentals of software engineering 2-4/lab 4/wildfly/jboss-modules.jar -mp /User...

Overview Memory Threads Classes VM Summary MBeans

Attribute values

Name	Value
AverageClickInterval	1000.5

Refresh

Left sidebar tree:

- JMImplementation
 - app.monitoring
 - ClickStats
 - Attributes
 - Operations
 - PointStats
 - com.sun.management
 - java.lang
 - java.nio
 - java.util.logging
 - jboss.as
 - jboss.as.expr
 - jboss.jta
 - jboss.modules
 - jboss.msc
 - jboss.remoting.endpoint
 - jboss.remoting.handler
 - jboss.root
 - jboss.threads
 - jboss.ws
 - jdk.management.jfr
 - org.wildfly.clustering
 - org.xnio

pid: 13570 /Users/sonikpav/ITMO/fundamentals of software engineering 2-4/lab 4/wildfly/jboss-modules.jar -mp /User...

Overview Memory Threads Classes VM Summary MBeans

Attribute values

Name	Value
OutsidePoints	2
TotalPoints	3

Refresh

Left sidebar tree:

- JMImplementation
 - app.monitoring
 - ClickStats
 - Attributes
 - Operations
 - PointStats
 - Attributes
 - Operations
 - Notifications
 - com.sun.management
 - java.lang
 - java.nio
 - java.util.logging
 - jboss.as
 - jboss.as.expr
 - jboss.jta
 - jboss.modules
 - jboss.msc
 - jboss.remoting.endpoint
 - jboss.remoting.handler
 - jboss.root
 - jboss.threads
 - jboss.ws
 - jdk.management.jfr
 - org.wildfly.clustering
 - org.xnio

Уведомления, приходящие от PointStatsMBean'a

The screenshot shows the JBoss IDE interface with the 'MBeans' tab selected. On the left, a tree view shows the package hierarchy, with 'PointStats' under 'app.monitoring' expanded. The main pane displays the 'Notification buffer' for 'PointStats'. It contains a table with the following data:

TimeStamp	Type	UserData	SeqNum	Message	Event	Source
21:06:57:181	point.outside		3	Point is outsi...	javax.manag...	app.monitoring:ty...
21:06:52:294	point.outside		2	Point is outsi...	javax.manag...	app.monitoring:ty...
21:06:50:836	point.outside		1	Point is outsi...	javax.manag...	app.monitoring:ty...

At the bottom of the notification buffer, there are three buttons: 'Subscribe', 'Unsubscribe', and 'Clear'.

Наименование и версия JVM, поставщик виртуальной машины Java и номер её сборки, время работы:

The screenshot shows the JBoss IDE interface with the 'VM Summary' tab selected. The main pane displays the 'VM Summary' for the current JVM. The summary includes the following information:

Connection name: pid: 14185 /Users/sonikpav/ITMO/fundamentals of software engineering 2-4/lab 4/wildfly/jboss-modules.jar -mp /User...

Uptime: 4 minutes

Process CPU time: 21,837 seconds

JIT compiler: HotSpot 64-Bit Tiered Compilers

Total compile time: 10,236 seconds

Virtual Machine: OpenJDK 64-Bit Server VM version 22+36-2370

Vendor: Oracle Corporation

Name: 14185@Noutbuk-Sofa.local

Live threads: 70

Peak: 153

Daemon threads: 28

Total threads started: 163

Current heap size: 79 627 kbytes

Maximum heap size: 524 288 kbytes

Committed memory: 155 648 kbytes

Pending finalization: 0 objects

Garbage collector: Name = 'G1 Young Generation', Collections = 61, Total time spent = 0,158 seconds

Garbage collector: Name = 'G1 Concurrent GC', Collections = 32, Total time spent = 0,050 seconds

Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0,000 seconds

Operating System: Mac OS X 14.6.1

Architecture: aarch64

Number of processors: 8

Committed virtual memory: 413 319 120 kbytes

Total physical memory: 8 388 608 kbytes

Free physical memory: 172 464 kbytes

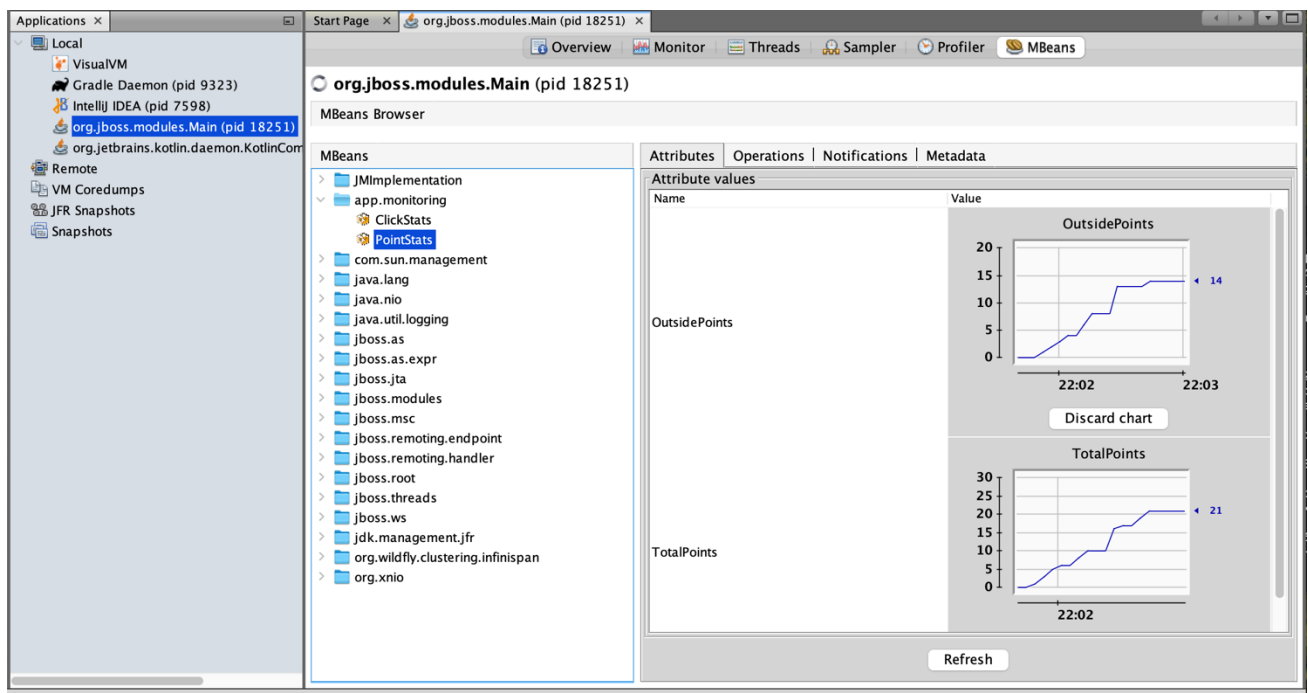
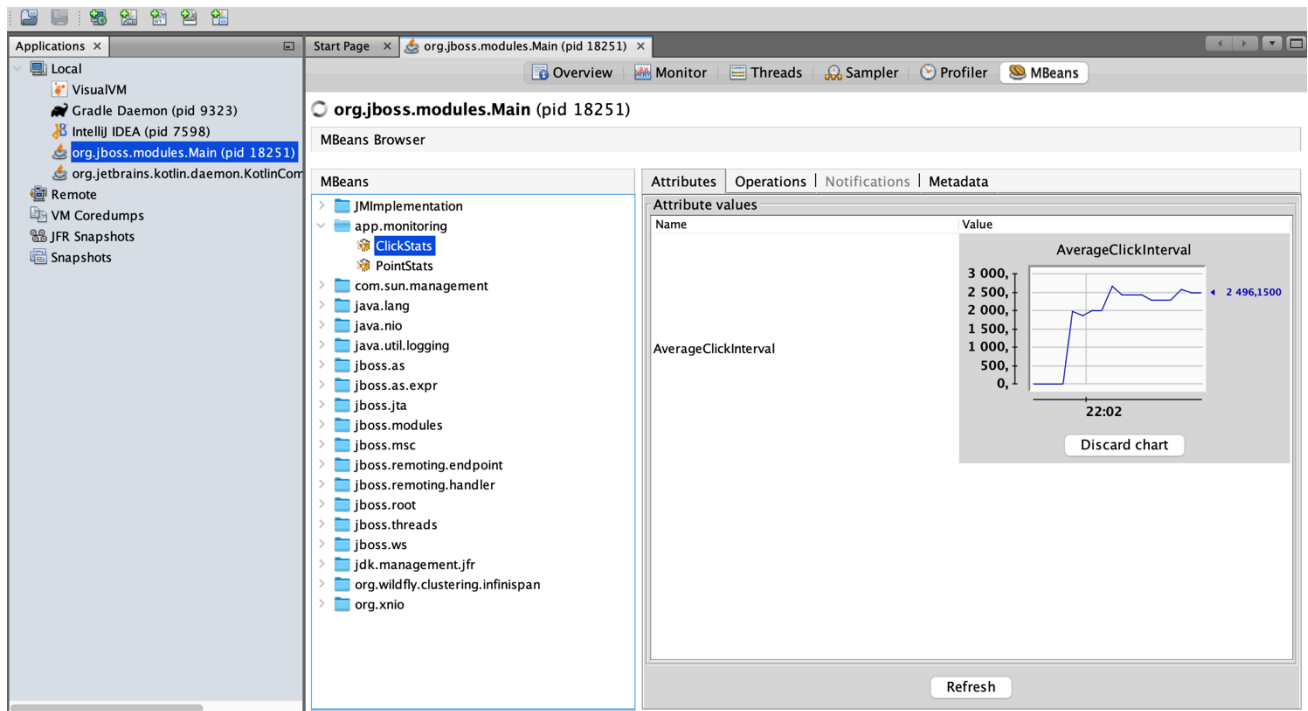
Total swap space: 2 097 152 kbytes

Free swap space: 454 912 kbytes

VM arguments: -D[Standalone] -Xms64m -Xmx512m -XX:MetaspaceSize=96M -XX:MaxMetaspaceSize=256m -Djava.net.preferIPv4Stack=true -Djboss.modules...

Задание 3

Графики изменения показаний MBean-классов.



Поток, потребляющий наибольший процент времени CPU (43%) – default task-1.

org.jboss.modules.Main (pid 10114)

Sampler ☐ Settings

Sample: ☒ CPU ☐ Memory ☐ Stop

Status: sampling inactive

CPU samples Thread CPU time

Results: ☐ ☒ ☐ Statistics: Threads Count: 71 Total Time (CPU): 59 156 ms Thread Dump

Name	Thread Time (CPU)	Thread Time (CPU) / sec
default task-1	25 506 ms (43,1 %)	0,0 ms (0 %)
RMI TCP Connection(13)-127.0.0.1	20 170 ms (34,1 %)	0,0 ms (0 %)
RMI TCP Connection(10)-127.0.0.1	9 373 ms (15,8 %)	93,4 ms (9,3 %)
MSC service thread 1-1	688 ms (1,2 %)	0,0 ms (0 %)
default I/O-4	527 ms (0,9 %)	0,0 ms (0 %)
DeploymentScanner-threads - 1	500 ms (0,8 %)	0,0 ms (0 %)
DestroyJavaVM	403 ms (0,7 %)	0,0 ms (0 %)
default I/O-2	374 ms (0,6 %)	0,704 ms (0,1 %)
MSC service thread 1-7	292 ms (0,5 %)	0,0 ms (0 %)
ServerService Thread Pool -- 81	279 ms (0,5 %)	0,0 ms (0 %)
MSC service thread 1-2	169 ms (0,3 %)	0,0 ms (0 %)
Attach Listener	129 ms (0,2 %)	0,0 ms (0 %)
MSC service thread 1-6	113 ms (0,2 %)	0,0 ms (0 %)
MSC service thread 1-5	111 ms (0,2 %)	0,0 ms (0 %)
Notification Thread	106 ms (0,2 %)	0,0 ms (0 %)
MSC service thread 1-3	89,6 ms (0,2 %)	0,0 ms (0 %)
MSC service thread 1-8	89,5 ms (0,2 %)	0,0 ms (0 %)
MSC service thread 1-4	69,9 ms (0,1 %)	0,0 ms (0 %)
DeploymentScanner-threads - 2	25,1 ms (0 %)	0,0 ms (0 %)

org.jboss.modules.Main (pid 10114)

Thread Dump

```
at java.lang.Thread.runWith(java.base@22/Thread.java:1503)
at java.lang.Thread.run(java.base@22/Thread.java:1570)
at org.jboss.threads.JBossThread.run(JBossThread.java:513)

Locked ownable synchronizers:
- None

"default task-1" #183 [55047] prio=5 os_prio=31 cpu=25447,86ms elapsed=592,69s tid=0x000000013d780600 nid=55047 waiting on c
java.lang.Thread.State: TIMED_WAITING (parking)
at jdk.internal.misc.Unsafe.park(java.base@22/Native Method)
- parking to wait for <0x00000007e41c7788> (a org.jboss.threads.EnhancedQueueExecutor)
at java.util.concurrent.locks.LockSupport.parkNanos(java.base@22/LockSupport.java:269)
at org.jboss.threads.EnhancedQueueExecutor$ThreadBody.run(EnhancedQueueExecutor.java:1421)
at org.xnio.XnioWorker$WorkerThreadFactory$1$1.run(XnioWorker.java:1282)
at java.lang.Thread.runWith(java.base@22/Thread.java:1583)
at java.lang.Thread.run(java.base@22/Thread.java:1570)

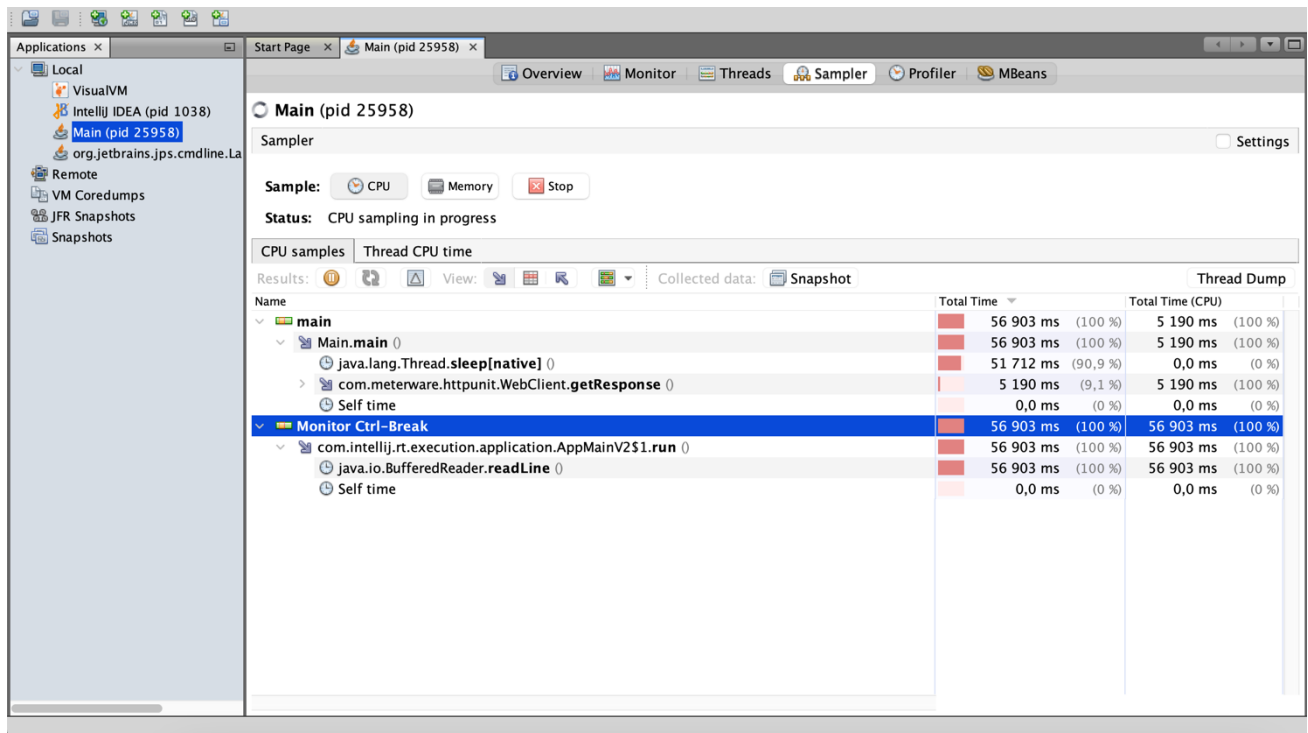
Locked ownable synchronizers:
- None

"PostgreSQL-JDBC-Cleaner" #184 [49927] daemon prio=5 os_prio=31 cpu=3,15ms elapsed=592,16s tid=0x0000000122493c00 nid=49927
java.lang.Thread.State: TIMED_WAITING (parking)
at jdk.internal.misc.Unsafe.park(java.base@22/Native Method)
- parking to wait for <0x00000007e27e3e58> (a java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
at java.util.concurrent.locks.LockSupport.parkNanos(java.base@22/LockSupport.java:269)
at java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject.await(java.base@22/AbstractQueuedSynchroniz
at java.lang.ref.ReferenceQueue.await(java.base@22/ReferenceQueue.java:71)
at java.lang.ref.ReferenceQueue.remove0(java.base@22/ReferenceQueue.java:143)
at java.lang.ref.ReferenceQueue.remove(java.base@22/ReferenceQueue.java:218)
```

Find: default task-1 1 of 1 matches

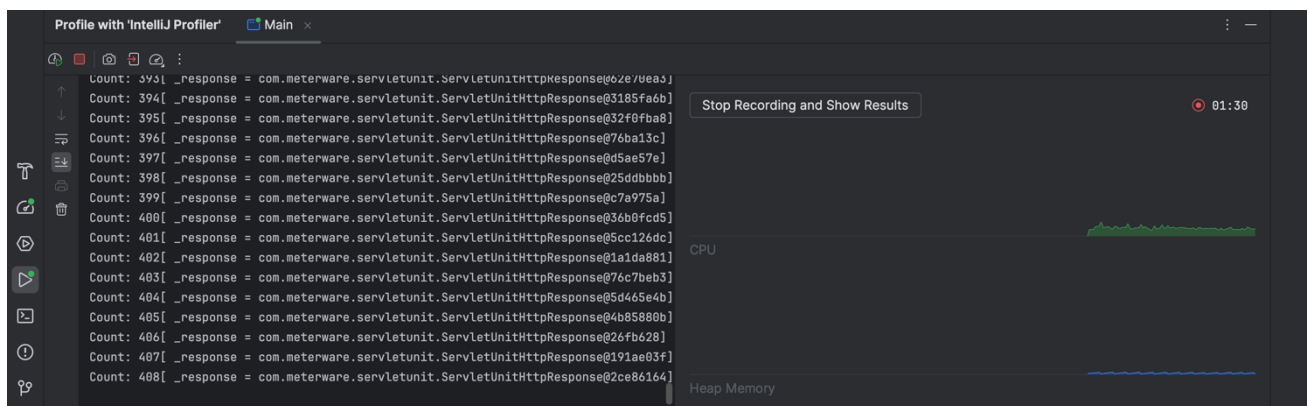
Задание 4

1. Запускаем Sampler CPU в VisualVM и IDEA Profiler:



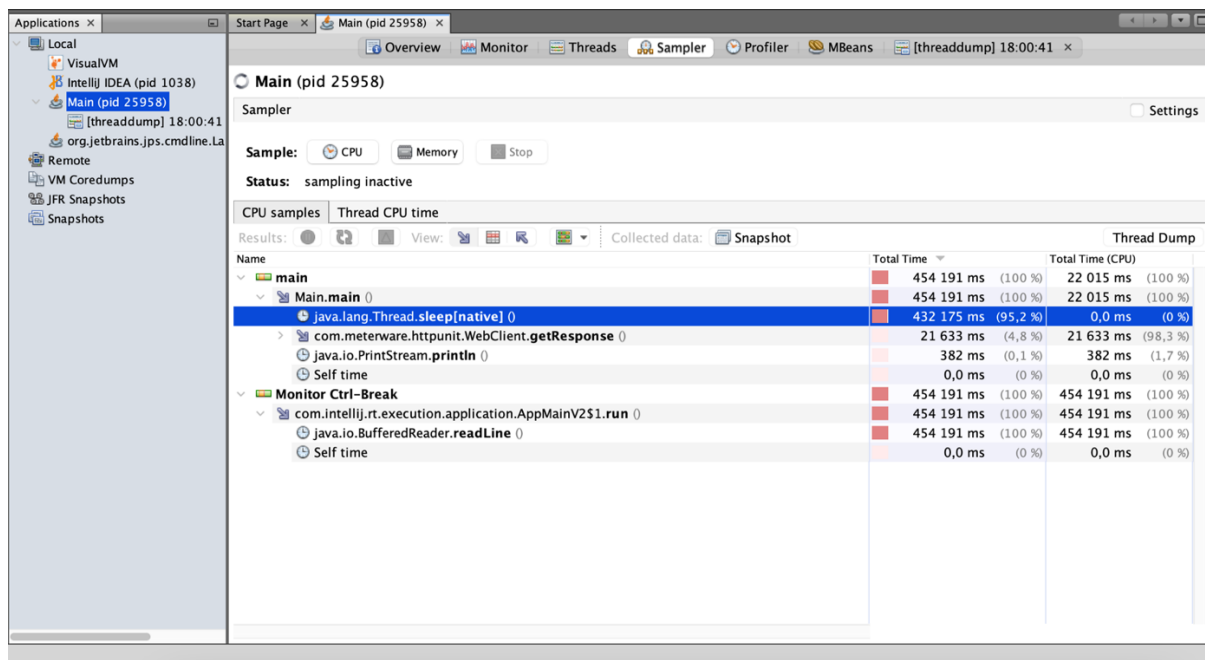
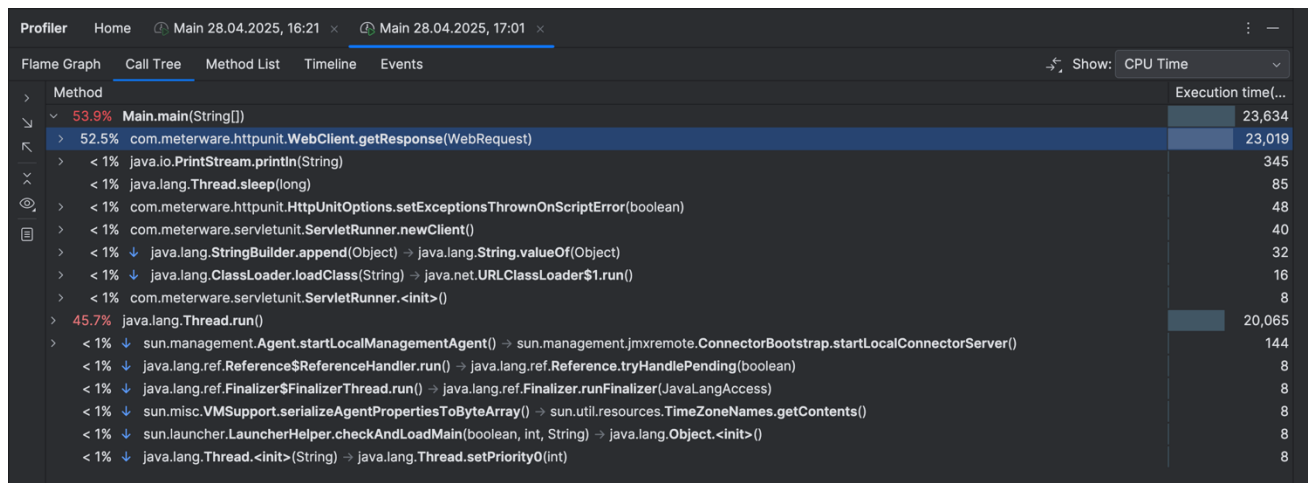
The screenshot shows the VisualVM interface with the 'Main (pid 25958)' application selected. The 'Sampler' tab is active, showing 'CPU' sampling in progress. The 'Thread CPU time' table displays the following data:

Name	Total Time	Total Time (CPU)
main	56 903 ms (100 %)	5 190 ms (100 %)
Main.main ()	56 903 ms (100 %)	5 190 ms (100 %)
java.lang.Thread.sleep[native] ()	51 712 ms (90,9 %)	0,0 ms (0 %)
com.meterware.httpunit.WebClient.getResponse ()	5 190 ms (9,1 %)	5 190 ms (100 %)
Self time	0,0 ms (0 %)	0,0 ms (0 %)
Monitor Ctrl-Break	56 903 ms (100 %)	56 903 ms (100 %)
com.intellij.rt.execution.application.AppMainV2\$1.run ()	56 903 ms (100 %)	56 903 ms (100 %)
java.io.BufferedReader.readLine ()	56 903 ms (100 %)	56 903 ms (100 %)
Self time	0,0 ms (0 %)	0,0 ms (0 %)



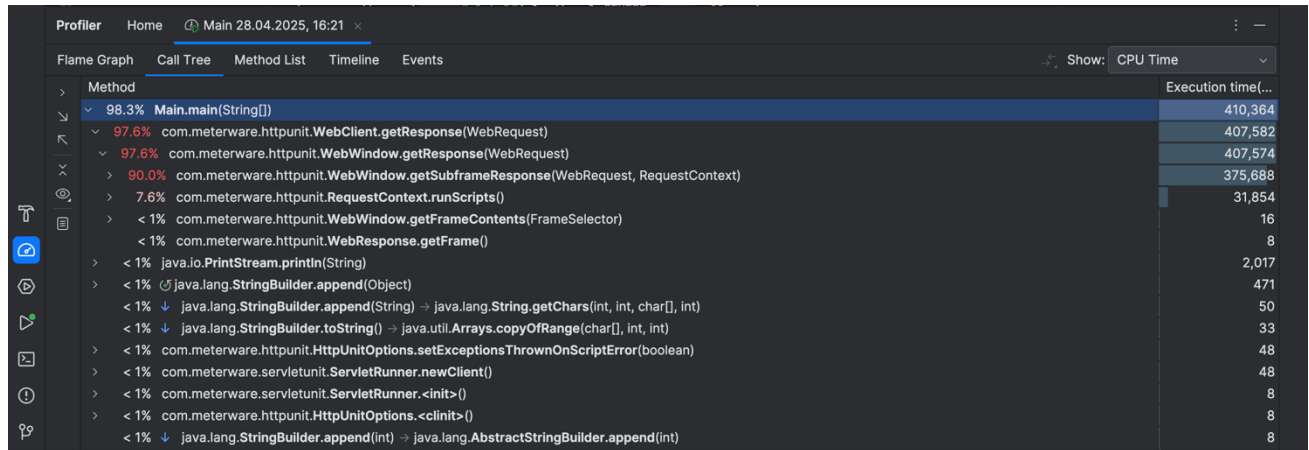
The screenshot shows the IntelliJ Profiler interface with the 'Main' application selected. The 'CPU' and 'Heap Memory' usage is displayed over time. The CPU usage is shown as a green line graph, and the Heap Memory usage is shown as a blue line graph. The CPU usage is high, indicating a CPU-intensive operation. The Heap Memory usage is low, indicating a memory-efficient operation. The 'Stop Recording and Show Results' button is visible.

2. После их 10-минутной работы проанализируем результаты:



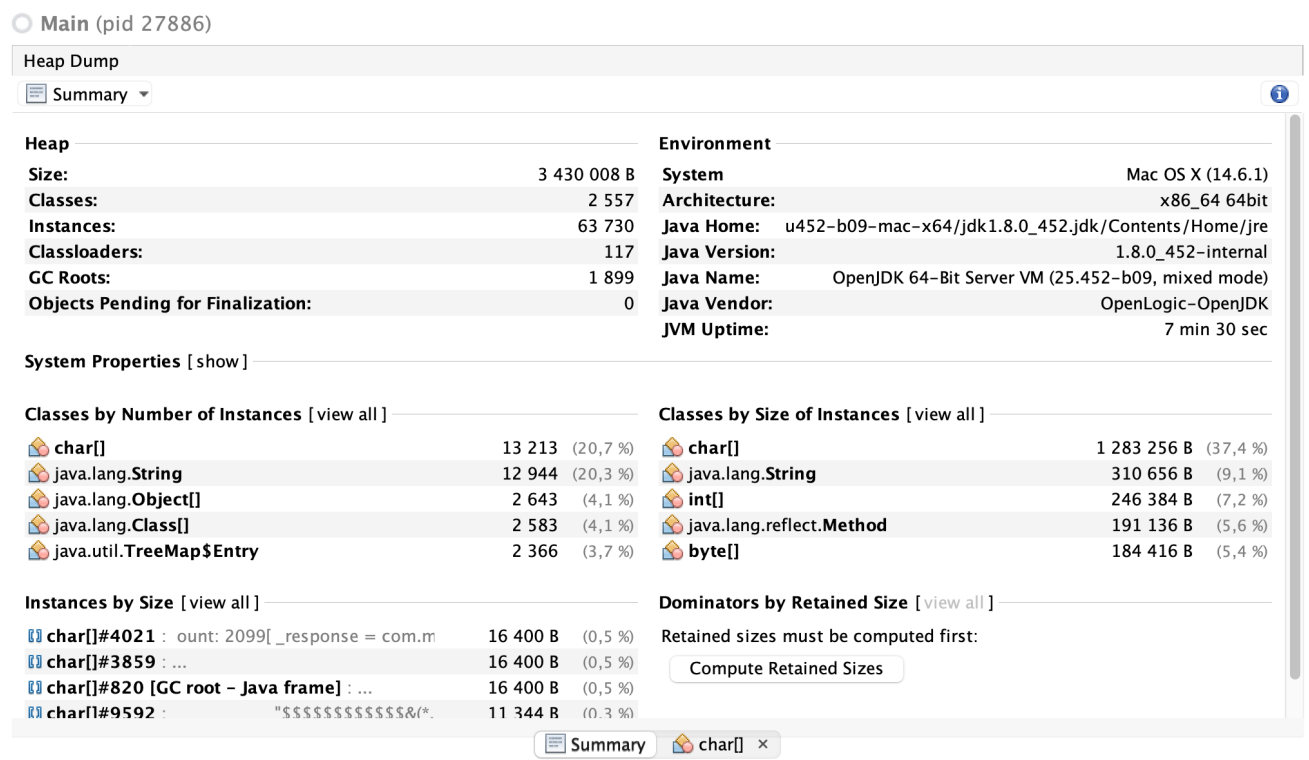
```
37 public static void main(String[] args) {
38     try {
39         HttpUnitOptions.setExceptionsThrownOnScriptError(false);
40         ServletRunner sr = new ServletRunner();
41         sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
42         ServletUnitClient sc = sr.newClient();
43         int number = 1;
44         WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
45         while (true) {
46             WebResponse response = sc.getResponse(request);
47             System.out.println("Count: " + number++ + response);
48             java.lang.Thread.sleep( millis: 200);
49         }
50     } catch (InterruptedException ex) {
51         Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
52     } catch (MalformedURLException ex) {
53         Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
54     } catch (IOException ex) {
55         Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
56     } catch (SAXException ex) {
57         Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
58     }
59 }
```

3. Наибольшее время CPU (53.9%) в нашем приложении уходит на метод `WebClient.getResponse()`. Заменить или убрать мы его не можем, так как в нем заключается основная работа нашего приложения. В цикле вместе с этим методом есть метод `Thread.sleep()`. Он не тратит время CPU, но не несет никакой функциональной нагрузки. Можем его удалить и проверить производительность заново:



Мы видим, что теперь `WebClient.getResponse()` тратит 98% от времени CPU, так как выполняется значительно чаще и без задержек.

4. Проверим приложение на утечки памяти, для этого запустим Sampler Memory в VisualVM. После 10-минутной работы снимаем Heap Dump. Видим, что большую часть памяти занимают объекты типа `char[]`.



○ Main (pid 27886)

Heap Dump			
char[] Details: Preview Fields References GC Root Hierarchy			
Name	Count	Size	Retained
char[]	13 213 (20,7 %)	1 283 256 B (37,4 %)	1 162 976 B (33,9 %)
> char[]#820 [GC root - Java frame] : ...		16 400 B (0,5 %)	16 400 B (0,5 %)
> char[]#3859 : ...		16 400 B (0,5 %)	16 400 B (0,5 %)
> char[]#4021 : out: 2099[_response = com.meterware.servletunit.Ser		16 400 B (0,5 %)	16 400 B (0,5 %)
> char[]#7895 : "\$\$\$\$\$\$\$\$\$\$(*,.....02468::<>@BDI		11 344 B (0,3 %)	11 344 B (0,3 %)
> char[]#9592 : "\$\$\$\$\$\$\$\$\$\$(*,.....02468::<>@BDI		11 344 B (0,3 %)	11 344 B (0,3 %)
> char[]#3642 : /Users/sonikpav/Downloads/openlogic-openjdk-8u45		5 712 B (0,2 %)	5 712 B (0,2 %)
> char[]#7536 : 0@P`p *ÅÐàð ÅÐÇIÍÓŠÚÛbEOuĴĴÄ Ñ Ĩ Ōzeđuf		4 112 B (0,1 %)	4 112 B (0,1 %)
> char[]#9616 : 0@P`p *ÅÐàð ÅÐÇIÍÓŠÚÛbEOuĴĴÄ Ñ Ĩ Ōzeđuf		4 112 B (0,1 %)	4 112 B (0,1 %)
> char[]#7767 : 蹣ꠘ蹣ꠘ蹣ꠘ蹣ꠘ堀匂匂匂匂匂匂匂匂匂匂匂匂匂匂匂匂匂匂匂		3 736 B (0,1 %)	3 736 B (0,1 %)
> char[]#10111 : sun.,com.sun.xml.internal.,com.sun.imageio.,com.sun		2 648 B (0,1 %)	2 648 B (0,1 %)
> char[]#10147 : sun.,com.sun.xml.internal.,com.sun.imageio.,com.sun		2 648 B (0,1 %)	2 648 B (0,1 %)
> char[]#11035 : ADANDAEAREAFAGAGATGAIAIALBAMARMANANT		2 520 B (0,1 %)	2 520 B (0,1 %)
> char[]#11034 : aaaarababkaeaveafafrakakaamamhanargararasas mave		1 896 B (0,1 %)	1 896 B (0,1 %)
> char[]#7419 : ...		1 880 B (0,1 %)	1 880 B (0,1 %)
> char[]#9534 : ...		1 880 B (0,1 %)	1 880 B (0,1 %)
> char[]#6941 : Êþ%3+ ! " # <init> ()V Code _invokeK(L		1 856 B (0,1 %)	1 856 B (0,1 %)
> char[]#3809 : /Users/sonikpav/Downloads/openlogic-openjdk-8u45		1 792 B (0,1 %)	1 792 B (0,1 %)
> char[]#7896 : en en-AU en-CA en-GB en-IE en-IN en-MT en-NZ en-		1 720 B (0,1 %)	1 720 B (0,1 %)
> char[]#7504 : en-AU en-CA en-GB en-IE en-IN en-MT en-NZ en-PH		1 440 B (0 %)	1 440 B (0 %)
> char[]#7502 : en en-AU en-CA en-GB en-IE en-IN en-MT en-NZ en-		1 432 B (0 %)	1 432 B (0 %)

5. Исследовав кучу, находим повторяющиеся строки, которые создаются нашим приложением (пакет com.meterware):

○ Main (pid 27886)

Heap Dump			
char[] Details: Preview Fields References GC Root Hierarchy			
Name	Count	Size	Retained
> char[]#10109 : (Ljava/lang/ClassLoader;(Ljava/lang/Class<*>;Ljava/lang/reflect/InvocationHandler;)Lj		224 B (0 %)	0 B (0 %)
> char[]#10641 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> <items>			
> <references>			
> value in java.lang.String#10387 : Script 'document.wr('Hello Document') //' failed: TypeError:		24 B (0 %)	248 B (0 %)
> [188] in java.lang.Object[]#2580 : 2 776 items	11 120 B (0,3 %)		531 672 B (15,5 %)
> elementData in java.util.ArrayList#75 : 2 099 elements	24 B (0 %)		531 696 B (15,5 %)
> static_errorMessages in class com.meterware.httpunit.javascrip.Javascript : JavaScript		64 B (0 %)	531 736 B (15,5 %)
> char[]#10907 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11173 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11306 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11572 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11705 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11838 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#11971 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12104 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12237 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12370 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12503 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12769 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)
> char[]#12902 : Script 'document.wr('Hello Document') //' failed: TypeError: undefined is not a function.		224 B (0 %)	224 B (0 %)

6. Найдем в коде место, где создаются строки:

```
public class JavaScript {

    private final static Object[] NO_ARGS = new Object[0]; 1 usage

    private static boolean _throwExceptionsOnError = true; 2 usages

    private static ArrayList _errorMessages = new ArrayList(); 4 usages

    static boolean isThrowExceptionsOnError() { return _throwExceptionsOnError; }

    static void setThrowExceptionsOnError( boolean throwExceptionsOnError ) { 1 usage
        _throwExceptionsOnError = throwExceptionsOnError;
    }

    static void clearErrorMessages() { _errorMessages.clear(); }
```

```
public void clearErrorMessages() { 1 usage
    JavaScript.clearErrorMessages();
}
```

```
/**
 * Clears the accumulated script error messages.
 */
public static void clearScriptErrorMessages() { no usages
    getScriptingEngine().clearErrorMessages();
}
```

7. Обнаруживаем, что у нас используется статический ArrayList() с ошибками, который не очищается, и тем самым засоряет память. Решением будет очистка массива после выполнения очередного запроса:

```
public static void main(String[] args) {
    try {
        HttpUnitOptions.setExceptionsThrownOnScriptError(false);
        ServletRunner sr = new ServletRunner();
        sr.registerServlet(resourceName: "myServlet", HelloWorld.class.getName());
        ServletUnitClient sc = sr.newClient();
        int number = 1;
        WebRequest request = new GetMethodWebRequest(urlString: "http://test.meterware.com/myServlet");
        while (true) {
            WebResponse response = sc.getResponse(request);
            System.out.println("Count: " + number++ + response);
            HttpUnitOptions.clearScriptErrorMessage();
        }
    } catch (MalformedURLException ex) {
        Logger.getLogger(name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (IOException ex) {
        Logger.getLogger(name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (SAXException ex) {
        Logger.getLogger(name: "global").log(Level.SEVERE, msg: null, ex);
    }
}
```

8. Еще раз запустим Sampler Memory в VisualVM:

○ Main (pid 2649)

Heap Dump

Summary

Heap			Environment		
Size:	2 611 808 B		System	Mac OS X (14.6.1)	
Classes:	2 555		Architecture:	x86_64 64bit	
Instances:	55 745		Java Home:	openjdk-8u452-b09-mac-x64/jdk1.8.0_452.jdk/Contents/Home/jre	
Classloaders:	117		Java Version:	1.8.0_452-internal	
GC Roots:	1 929		Java Name:	OpenJDK 64-Bit Server VM (25.452-b09, mixed mode)	
Objects Pending for Finalization:	0		Java Vendor:	OpenLogic-OpenJDK	
			JVM Uptime:	4 min 57 sec	

System Properties [show]

Classes by Number of Instances [view all]			Classes by Size of Instances [view all]		
char[]	10 987	(19,7 %)	char[]	805 792 B	(30,9 %)
java.lang.String	10 716	(19,2 %)	java.lang.String	257 184 B	(9,8 %)
java.lang.Class[]	2 583	(4,6 %)	byte[]	201 136 B	(7,7 %)
java.lang.Object[]	2 509	(4,5 %)	java.lang.reflect.Method	191 136 B	(7,3 %)
java.lang.reflect.Method	2 172	(3,9 %)	java.lang.Object[]	129 520 B	(5 %)

Instances by Size [view all]			Dominators by Retained Size [view all]		
char[]#2238 : ...	16 400 B	(0,6 %)	Retained sizes must be computed first:		
char[]#2259 : ount: 1266[_response = com.meterware.s	16 400 B	(0,6 %)	Compute Retained Sizes		
char[]#445 [GC root - Java frame] : ...	16 400 B	(0,6 %)			
char[]#9239 : "SSSSSSSSSSSSS&(*.....02	11 344 B	(0,4 %)			

Summary char[]

Заключение

В ходе лабораторной работы были разработаны и подключены два MBean-класса для подсчета общего числа точек и процента промахов пользователя на координатной плоскости, реализована отправка уведомлений. С помощью JConsole и VisualVM проведено мониторинг приложения: зафиксированы показатели MBean-ов, определен поток, потребляющий наибольший процент времени CPU. С помощью VisualVM и IDEA Profiler выявлены проблемы с производительностью в приложении HttpUnit, такие как лишние задержки и утечки памяти из-за неочищаемых коллекций. После оптимизации приложение стало работать быстрее и стабильнее, что подтверждено повторным профилированием.